

AI Assisted Coding

Name: P.VIKAS

Roll Number: 2403A510E6

Batch: 24BTCAICSB05

Task Description #1

The screenshot displays a Google Colab notebook interface. The main code cell contains a function `calculate_stats` that takes a list of numbers and returns its mean, minimum, and maximum values. The function is tested with two example lists: `[10, 20, 30, 40, 50]` and an empty list `[]`. The output shows the mean for the first list as 30.0. A right-hand sidebar displays release notes for Colab, dated 2025-07-22, mentioning new features like the Gemini 2.5 Flash model and the Colab AI library. The bottom status bar indicates the notebook is running on Python 3.

```
mean = sum(numbers) / len(numbers)
minimum = min(numbers)
maximum = max(numbers)

return mean, minimum, maximum

# Example usage:
my_list = [10, 20, 30, 40, 50]
mean_val, min_val, max_val = calculate_stats(my_list)

print(f"List: {my_list}")
print(f"Mean: {mean_val}")
print(f"Minimum: {min_val}")
print(f"Maximum: {max_val}")

empty_list = []
mean_val_empty, min_val_empty, max_val_empty = calculate_stats(empty_list)
print(f"\nEmpty list: {empty_list}")
print(f"Mean: {mean_val_empty}")
print(f"Minimum: {min_val_empty}")
print(f"Maximum: {max_val_empty}")
```

List: [10, 20, 30, 40, 50]
Mean: 30.0

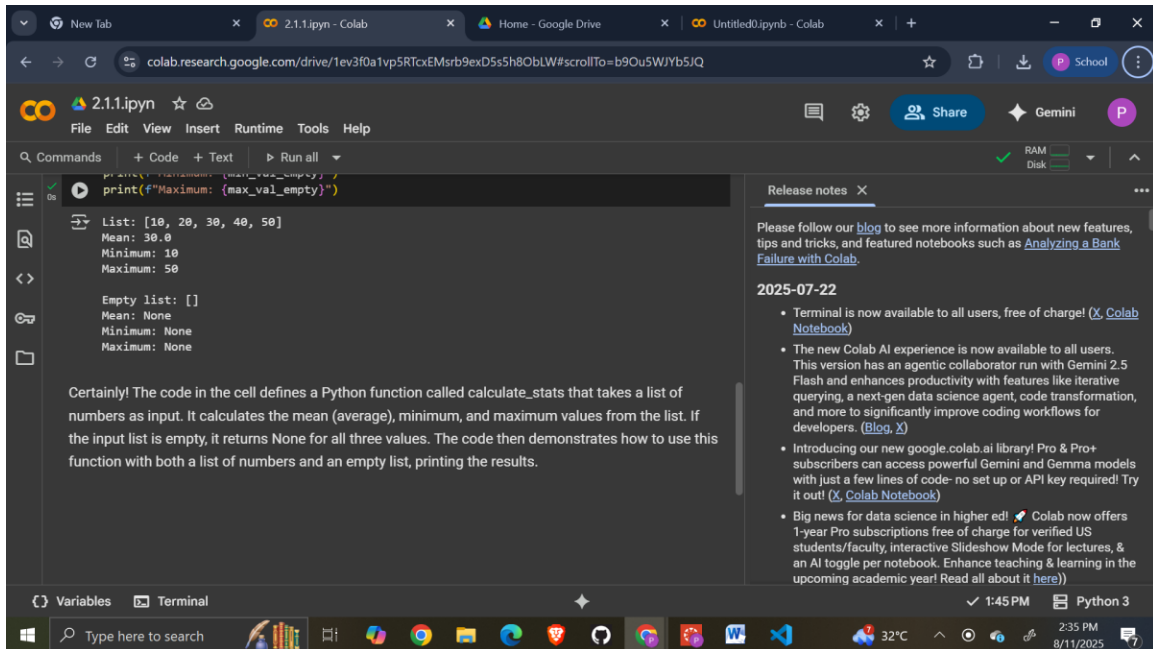
Release notes

Please follow our [blog](#) to see more information about new features, tips and tricks, and featured notebooks such as [Analyzing a Bank Failure with Colab](#).

2025-07-22

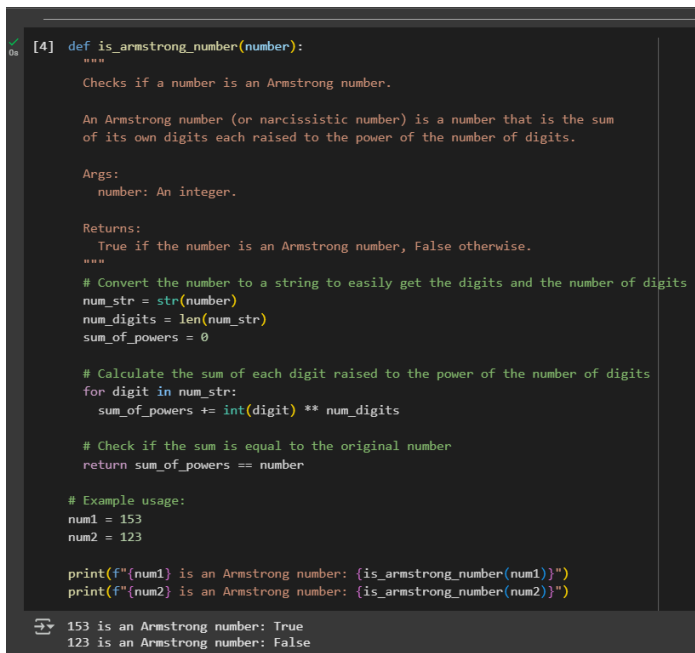
- Terminal is now available to all users, free of charge! ([X Colab Notebook](#))
- The new Colab AI experience is now available to all users. This version has an agentic collaborator run with Gemini 2.5 Flash and enhances productivity with features like iterative querying, a next-gen data science agent, code transformation, and more to significantly improve coding workflows for developers. ([Blog](#), [X](#))
- Introducing our new `google.colab.ai` library! Pro & Pro+ subscribers can access powerful Gemini and Gemma models with just a few lines of code- no set up or API key required! Try it out! ([X Colab Notebook](#))
- Big news for data science in higher ed! 🎉 Colab now offers 1-year Pro subscriptions free of charge for verified US students/faculty, interactive Slideshow Mode for lectures, & an AI toggle per notebook. Enhance teaching & learning in the upcoming academic year! Read all about it [here](#)!))

Variables Terminal 1:45 PM Python 3



Task Description #2

GEMINI:



CO-PILOT:

Python

Copy

```
def is_armstrong(number):
    # Convert the number to a string to easily iterate over digits
    digits = str(number)
    num_digits = len(digits)

    # Calculate the sum of each digit raised to the power of num_digits
    total = sum(int(digit) ** num_digits for digit in digits)

    # Check if the total equals the original number
    return total == number

# Example usage
print(is_armstrong(153)) # True
print(is_armstrong(9474)) # True
print(is_armstrong(123)) # False
```

How It Works

For 153 :

$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153 \rightarrow$ Armstrong number

For 9474 :

$9^4 + 4^4 + 7^4 + 4^4 = 9474 \rightarrow$ Armstrong number

For 123 :

$1^3 + 2^3 + 3^3 = 36 \neq 123 \rightarrow$ Not an Armstrong number

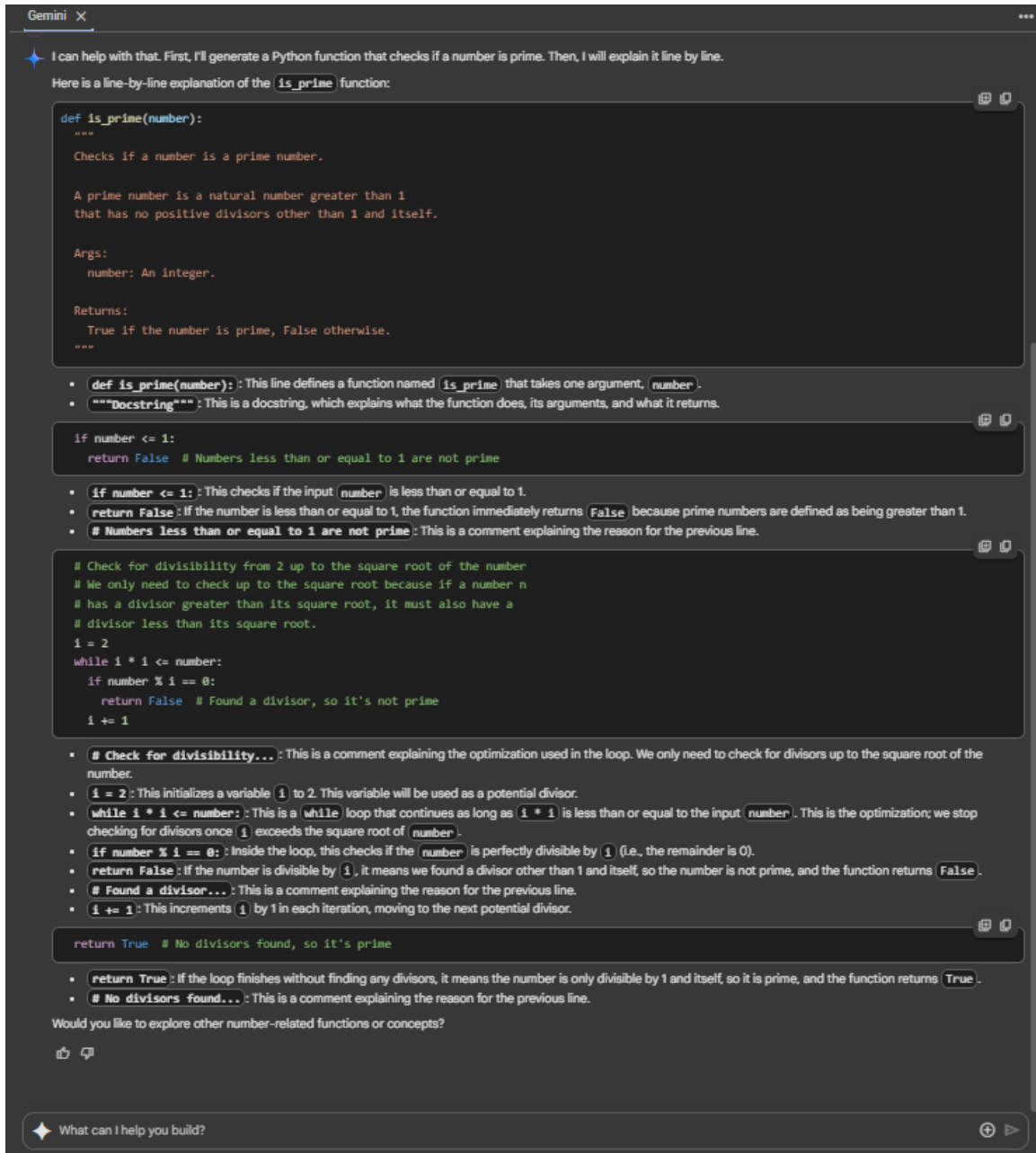
COMPARISION:

Feature / Step	Gemini Output (Screenshot 1)	Copilot Output (Screenshot 2)	Observations
Prompt Interpretation	Implements <code>is_armstrong_number(number)</code> with detailed docstring explaining Armstrong numbers, arguments, and return value.	Implements <code>is_armstrong(number)</code> with concise inline comments, skipping formal docstring.	Gemini focuses on educational clarity with docstrings; Copilot is more concise and direct.
Conversion to String	<code>num_str = str(number)</code> and separately counts digits: <code>num_digits = len(num_str)</code> .	Similar approach with <code>digits = str(number)</code> and <code>num_digits = len(digits)</code> .	Both approaches are functionally identical here.
Power Sum Calculation	Uses a <code>for</code> loop: <code>sum_of_powers += int(digit) ** num_digits</code> .	Uses a generator inside <code>sum()</code> : <code>sum(int(digit) ** num_digits for digit in digits)</code> .	Copilot uses a more Pythonic, compact one-liner; Gemini uses step-by-step clarity.
Return Condition	<code>return sum_of_powers == number</code>	<code>return total == number</code>	Identical logic; only variable naming differs.
Example Usage	Prints Armstrong check for 153 (True) and 123 (False).	Prints Armstrong check for 153 (True), 9474 (True), and 123 (False).	Copilot tests an additional well-known Armstrong number (9474).
Extra Explanation	Only outputs print results, no explanation of how it works.	Provides a "How It Works" section breaking down each example step-by-step.	Copilot's explanation makes the concept visually intuitive, with symbols and checks.
Readability	Beginner-friendly with clear steps and docstring.	Compact, elegant, with visual explanation for quick understanding.	Gemini suits detailed learning; Copilot suits quick reference.

Task Description #3

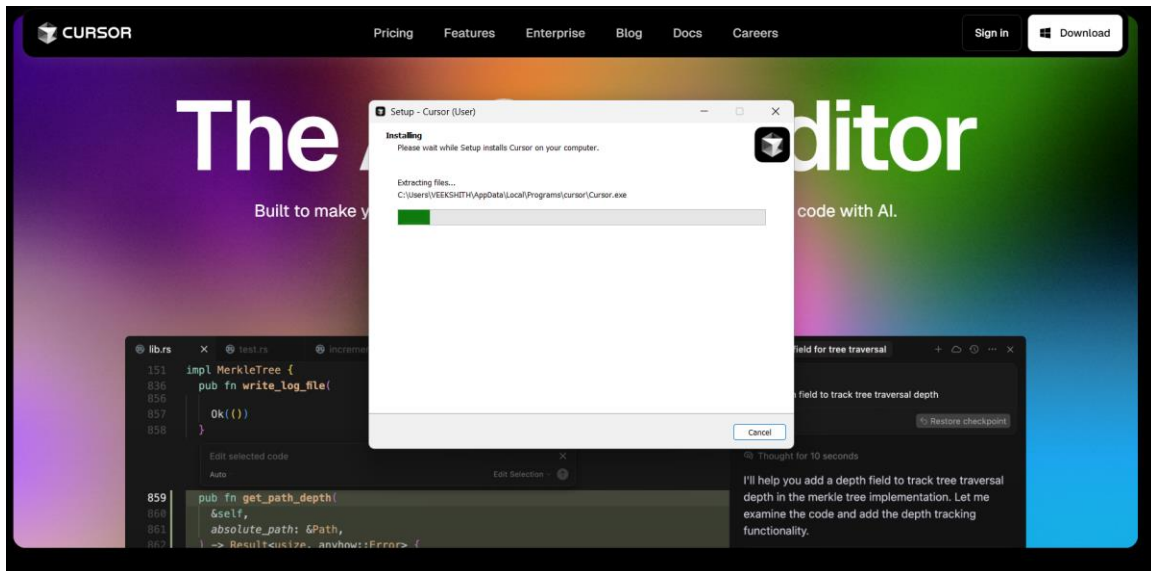
```
0s  def is_prime(number):  
    """  
    Checks if a number is a prime number.  
  
    A prime number is a natural number greater than 1  
    that has no positive divisors other than 1 and itself.  
  
    Args:  
        number: An integer.  
  
    Returns:  
        True if the number is prime, False otherwise.  
    """  
    if number <= 1:  
        return False # Numbers less than or equal to 1 are not prime  
  
    # Check for divisibility from 2 up to the square root of the number  
    # We only need to check up to the square root because if a number n  
    # has a divisor greater than its square root, it must also have a  
    # divisor less than its square root.  
    i = 2  
    while i * i <= number:  
        if number % i == 0:  
            return False # Found a divisor, so it's not prime  
        i += 1  
  
    return True # No divisors found, so it's prime  
  
# Example usage:  
num_to_check1 = 11  
num_to_check2 = 15  
  
print(f"{num_to_check1} is prime: {is_prime(num_to_check1)}")  
print(f"{num_to_check2} is prime: {is_prime(num_to_check2)}")  
  
 11 is prime: True  
15 is prime: False
```

LINE BY LINE EXPLANATION:



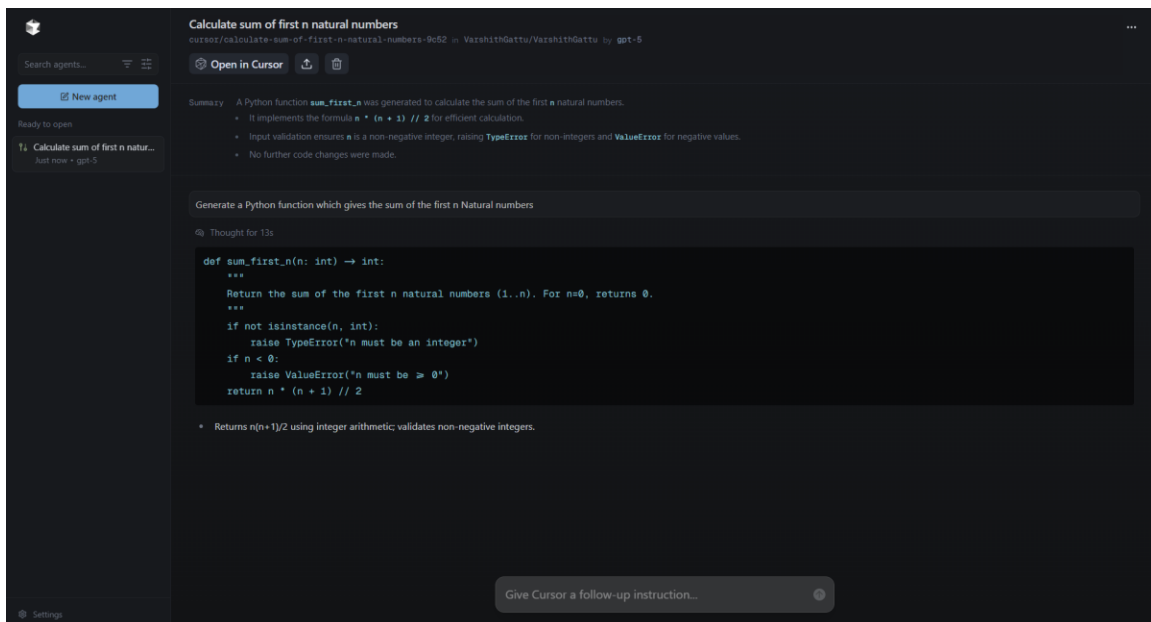
Task Description #4

Cursor AI setup:



Prompts used:

Generate a Python function which gives the sum of the first n Natural numbers.



Task Description #5

Original basic code:

```
numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
even_sum = 0
```

```
odd_sum = 0
```

```
for num in numbers:
```

```
    if num % 2 == 0:
```

```
        even_sum += num
```

```
    else:
```

```
        odd_sum += num
```

```
print("Sum of even numbers:", even_sum)
```

```
print("Sum of odd numbers:", odd_sum)
```

Sample Output:

Sum of even numbers: 30

Sum of odd numbers: 25

Refactored code:

```
def sum_even_odd(nums):  
  
    even_sum = sum(n for n in nums if n % 2 == 0)  
    odd_sum = sum(n for n in nums if n % 2 != 0)  
    return even_sum, odd_sum  
  
numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)  
  
even_total, odd_total = sum_even_odd(numbers)  
  
print(f"Sum of even numbers: {even_total}")  
print(f"Sum of odd numbers: {odd_total}")
```

Sample Output:

Sum of even numbers: 30

Sum of odd numbers: 25

Explanation of refactored code:

Line 1: Comment describing what the code does — calculates the sum of odd and even numbers in a tuple.

Line 3: Defines a function `sum_even_odd` with one parameter `nums`, which is expected to be a tuple of integers.

Line 4: A **docstring** explaining that the function returns the sum of even and odd numbers from the given tuple.

Line 5: Uses the `sum()` function with a generator expression to add up all numbers `n` in `nums` that are divisible by 2 (`n % 2 == 0`), meaning they are even.

Line 6: Similarly, sums up all numbers `n` in `nums` that are not divisible by 2 (`n % 2 != 0`), meaning they are odd.

Line 7: Returns both `even_sum` and `odd_sum` as a tuple.

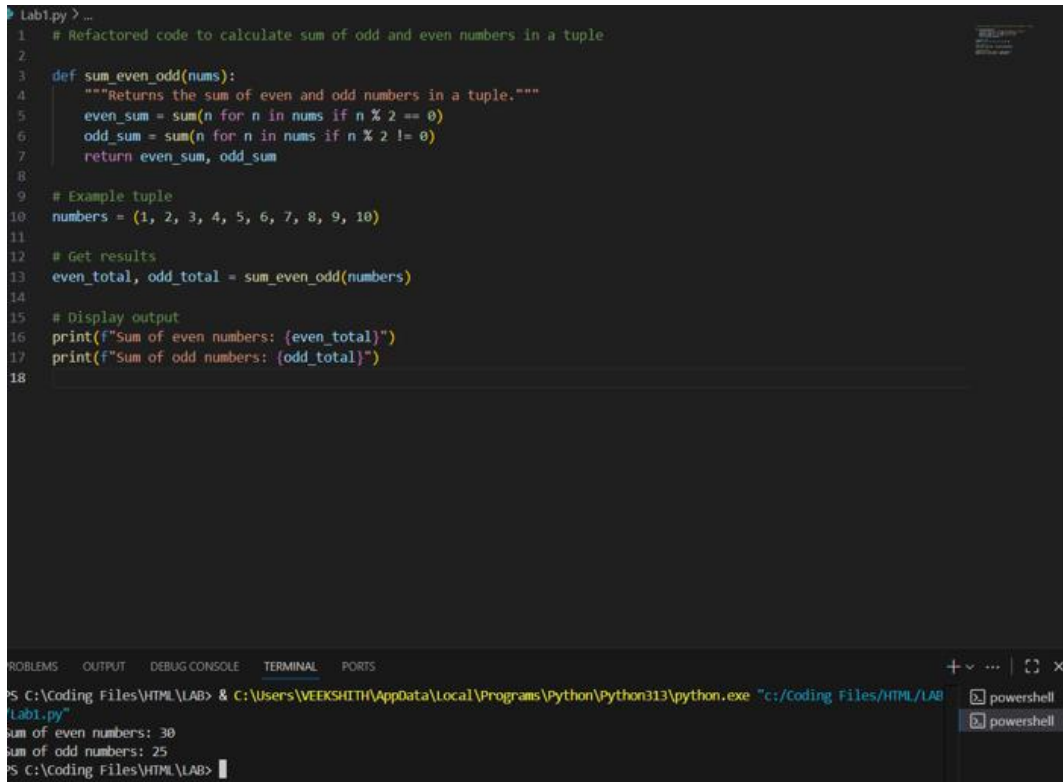
Line 10: Creates a sample tuple `numbers` containing integers from 1 to 10.

Line 13: Calls `sum_even_odd(numbers)` and stores the two returned values in `even_total` and `odd_total`.

Line 16: Prints the sum of even numbers using an f-string for clear formatting.

Line 17: Prints the sum of odd numbers using an f-string.

Output Screenshot:



The screenshot shows a Visual Studio Code editor with a Python file named 'Lab1.py'. The code defines a function 'sum_even_odd' that calculates the sum of even and odd numbers in a tuple. It then uses this function to calculate the sum of even and odd numbers for a specific tuple and prints the results using f-strings. The terminal output shows the execution of the script, displaying the sum of even numbers as 30 and the sum of odd numbers as 25.

```
1 # Refactored code to calculate sum of odd and even numbers in a tuple
2
3 def sum_even_odd(nums):
4     """Returns the sum of even and odd numbers in a tuple."""
5     even_sum = sum(n for n in nums if n % 2 == 0)
6     odd_sum = sum(n for n in nums if n % 2 != 0)
7     return even_sum, odd_sum
8
9 # Example tuple
10 numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
11
12 # Get results
13 even_total, odd_total = sum_even_odd(numbers)
14
15 # Display output
16 print(f"Sum of even numbers: {even_total}")
17 print(f"Sum of odd numbers: {odd_total}")
18
```

Terminal Output:

```
PS C:\Coding Files\HTML\LAB> & C:\Users\WEEKSHITH\AppData\Local\Programs\Python\Python313\python.exe "c:/Coding Files/HTML/LAB/Lab1.py"
sum of even numbers: 30
sum of odd numbers: 25
PS C:\Coding Files\HTML\LAB>
```